CÓDIGOCLARO · RUTA KOTLIN

# Tema 04 · Condiciones y when

Síntesis de estudio · Fuentes primarias enlazadas · DAM/DAW

## 1. Objetivo

if como expresión, when, rangos y smart casts.

### Objetivos de aprendizaje

- Explicar el propósito de condiciones y when con un ejemplo propio.
- Reconocer las condiciones que exige la solución y qué resultado debe producir.
- Contrastar las decisiones de sintaxis con la documentación primaria enlazada.
- Probar un caso normal y un caso límite antes de dar el ejercicio por terminado.

### Cómo estudiar este tema

Lee primero la explicación y separa los hechos del lenguaje de las decisiones del ejemplo. Predice el resultado antes de ejecutar el código. Después cambia un solo dato, anota qué esperas que ocurra y comprueba si la evidencia confirma tu razonamiento.

## 2. Base conceptual

if puede ser una expresión que devuelve un valor. `when` permite comparar con valores, rangos, tipos o condiciones, y puede usarse como expresión exhaustiva.

### Modelo mental

Para razonar sobre condiciones y when, sigue la relación entre el dato de entrada, la regla que describe la documentación, el estado que puede cambiar y el resultado observable. Explica qué parte del comportamiento garantiza el lenguaje y qué parte depende de las decisiones del programa.

### Vocabulario de trabajo

- Contrato: entradas admitidas y resultado que debe garantizarse.
- Estado: información que conserva el programa entre operaciones.
- Caso límite: una frontera que puede cambiar el resultado esperado.
- Regresión: un comportamiento correcto que deja de funcionar después de un cambio.

### Qué debes reconocer

- Identifica qué parte de condiciones y when resuelve el problema descrito en el objetivo.
- Distingue la regla del lenguaje de las APIs que aporta el entorno de ejecución.
- Antes de ejecutar el ejemplo, predice su resultado y explica qué dato lo determina.

### Preguntas antes de ejecutar

- ¿Qué línea representa directamente condiciones y when?
- ¿Qué valor o condición determina el resultado?
- ¿Qué ocurriría con una entrada vacía, repetida o fuera del rango esperado?

## 3. Ejemplo de referencia

```
val etiqueta = when (nota) {
    in 9..10 -> "Sobresaliente"
    in 7..8 -> "Notable"
    else -> "Revisar"
}
```

### Lectura del ejemplo

- Localiza la declaración o punto de entrada y nombra los datos que recibe.
- Señala la operación que aplica la idea central del tema.
- Anota el resultado esperado; después, ejecútalo y contrasta la predicción.
- Cambia una sola entrada y explica qué comportamiento debe variar.

# 4. Decisiones de diseño

Elige condiciones y when cuando el contrato del problema coincida con su propósito. Escribe primero qué entra, qué resultado esperas y qué restricciones deben seguir siendo ciertas. La descripción de este tema sirve como alcance; no presupongas capacidades que no aparezcan en la fuente de la versión utilizada.

### Descomposición del problema

- Entrada: identifica quién proporciona el dato y qué formato debe tener.
- Precondición: escribe qué debe ser cierto antes de aplicar la regla.
- Proceso: separa cada transformación y evita ocultar decisiones distintas.
- Salida: define el valor, cambio de estado o respuesta que se puede observar.
- Fallo: decide qué hacer si la entrada es inválida o una dependencia no responde.
- Coste: comprueba si el tamaño de los datos puede afectar tiempo o memoria.

### Comprobaciones que importan

- Caso normal: usa una entrada válida y registra el resultado observable.
- Límite: prueba el valor mínimo, máximo, vacío o repetido que tenga sentido para este tema.
- Error: decide cómo se comunica un dato inválido o un fallo externo.
- Compatibilidad: registra versión del lenguaje, runtime y biblioteca utilizada.
- Mantenimiento: conserva una prueba reproducible para el comportamiento que acabas de estudiar.

# 5. Precisión y límites

Comprueba el contrato del ejemplo con datos normales, límites y errores; consulta la documentación enlazada para confirmar la versión que usas.

### Depuración y diagnóstico

- Reproduce el fallo con la entrada más pequeña que todavía lo provoca.
- Lee el mensaje de error y localiza la primera línea relevante de la traza.
- Inspecciona los valores justo antes de la operación que no se comporta como esperabas.
- Formula una hipótesis y cambia una sola cosa en cada intento.
- Convierte el caso que fallaba en una prueba de regresión.
- Vuelve a ejecutar el caso normal y el límite para comprobar que no has roto otro comportamiento.

# 6. Práctica guiada

Construye una solución pequeña para practicar condiciones y when en una aplicación de aula. Usa la descripción «if como expresión, when, rangos y smart casts.» como requisito funcional y escribe antes una entrada y una salida esperadas.

- Implementa primero el caso normal con el ejemplo de la página anterior.
- Añade un caso límite y un caso que deba rechazarse o tratarse de otra forma.
- Separa los datos de prueba de la operación para poder repetir el ejercicio.
- Anota qué afirmación técnica has contrastado en la documentación oficial.

### 7. Verificación

| Prueba | Qué debe quedar claro |
| --- | --- |
| Normal | La regla principal produce el resultado previsto. |
| Límite | El comportamiento en la frontera está definido. |
| Error | El fallo no se oculta ni daña el estado. |

# 8. Laboratorio

Resuelve estas tareas sin copiar el ejemplo. En cada respuesta, explica qué aporta condiciones y when y qué evidencia usarías para comprobar la solución.

### Nivel inicial

Reproduce el ejemplo con nombres y datos nuevos. Mantén el objetivo «if como expresión, when, rangos y smart casts.» y escribe el resultado que esperas antes de ejecutarlo.

### Nivel intermedio

Amplía el caso con una segunda condición de negocio. Añade pruebas para el caso normal, un límite y un error; revisa que cada prueba comprueba un resultado observable.

### Nivel de aplicación

Integra condiciones y when en una pequeña función, componente o consulta del proyecto. Justifica una decisión de diseño con la referencia oficial del tema.

### Revisión por otra persona

Entrega el ejercicio sin explicar de antemano cómo funciona. Pide a otra persona que identifique la entrada, prediga el resultado y proponga un caso límite. Si necesita una explicación oral para entender el código, mejora los nombres o la estructura.

## 9. Solución orientativa y autoevaluación

Una respuesta sólida explica la regla aplicada, los datos de entrada, el resultado esperado y la condición que podría romper el caso. Debe distinguir la sintaxis documentada de las decisiones propias del ejercicio y citar la fuente que confirma el comportamiento.

### Criterios de revisión

| Criterio | En proceso | Consolidado |
| --- | --- | --- |
| Corrección | Solo resuelve el caso básico. | Incluye límites y errores relevantes. |
| Claridad | La intención no está clara. | Nombres y estructura muestran el propósito. |
| Evidencia | No hay resultado reproducible. | La prueba y la fuente justifican la respuesta. |

- ¿Puedes explicar condiciones y when sin repetir el ejemplo?
- ¿Qué caso límite probaste y por qué era relevante?
- ¿Qué parte depende de la versión o de una biblioteca externa?
- ¿Qué referencia oficial consultarías ante un resultado distinto?

### Glosario de repaso

Condiciones y when: tema y decisión técnica que se estudian en esta guía. Contrato: entradas admitidas y resultado prometido. Caso límite: frontera que puede cambiar la salida. Fuente primaria: documentación publicada o mantenida por el proyecto responsable del lenguaje o herramienta.

### Entrega final

Incluye un ejemplo mínimo, los datos de entrada, el resultado esperado y una prueba que se pueda volver a ejecutar. Anota la versión del lenguaje y pega la referencia exacta que confirma la regla aplicada. Si el resultado no coincide con tu predicción, registra el caso antes de cambiar el código.

## 10. Fuentes y atribución

Esta guía es una síntesis didáctica en español basada en documentación mantenida por JetBrains. No reproduce capítulos completos ni sustituye la referencia oficial; comprueba en los enlaces la versión de tu proyecto. Consulta:

JetBrains: Kotlin Documentation

JetBrains: Kotlin Standard Library